Legal AI Platform

Complete Project Understanding Report

Assistant-first explanation, backend deep dive, and full inventory

Generated: 2026-04-20 15:14

Workspace: C:\Users\ahmed\Desktop\pfe.2\legal-ai-platform

What this PDF gives you:

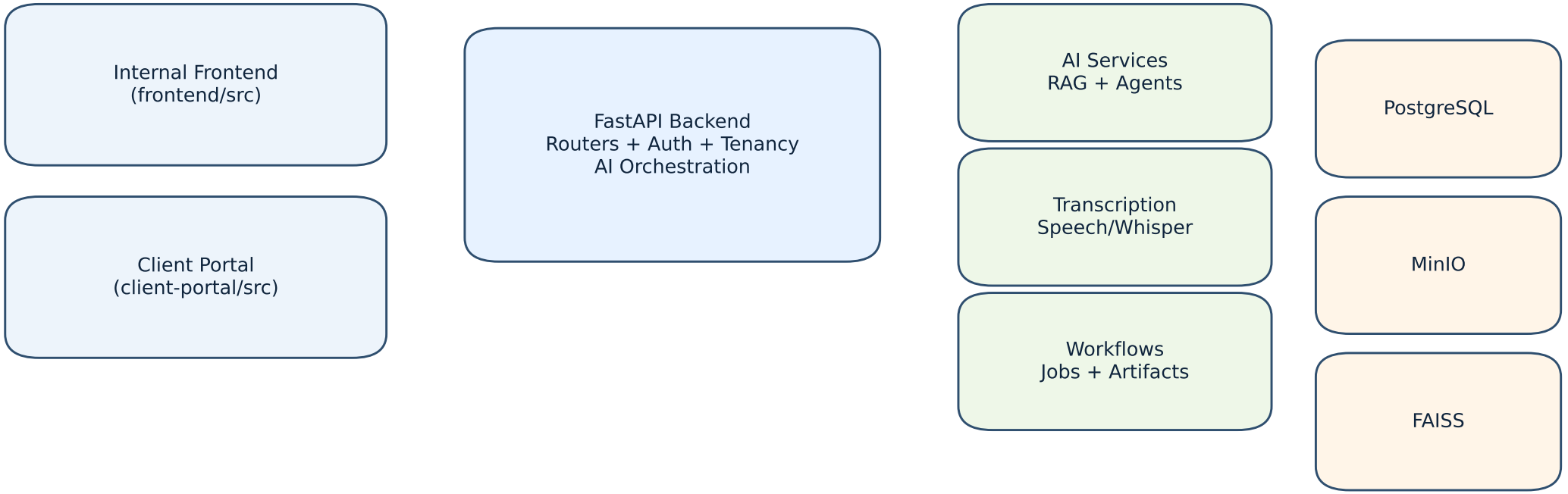
- A practical explanation of what your assistant does and why your architecture is strong.
- A backend-heavy walkthrough from request entry to AI response and persistence.
- A lighter frontend summary so you can understand user flow and API integration.
- A complete appendixed inventory of agents, endpoints, models, schemas, and intents.

Quick size snapshot:

Endpoints: 87 | Agents: 21 | AI modules: 40 | Models: 23 | Schemas: 121 | Intents: 42

Project Big Picture

What exists today and how major parts connect



Current Implementation Snapshot

- Backend startup includes schema initialization, background worker start, request-id middleware, and legacy schema patching
- Backend API inventory includes 87 detected endpoints across multiple domains (auth, cases, documents, AI/copilot, search, voice, intake).
- AI runtime includes 21 agent classes and 40 AI service modules, with 42 command intents parsed from natural language.
- Persistence layer includes 23 SQLAlchemy models and 121 Pydantic API schemas.

Why this architecture is good

- Strong separation of concerns: API, orchestration, intent execution, retrieval, and data layers are split into

Assistant Deep Dive

How your copilot works from user prompt to grounded answer

Assistant execution pipeline (orchestrated)

- 1) Context resolution: merges conversation history with persisted copilot memory.
- 2) Prompt correction: improves noisy prompts while preserving intent.
- 3) Intent detection: parses scope/target and intent class from natural language.
- 4) Low-confidence arbitration: stabilizes routing when parser confidence is weak or conflicting.
- 5) Prompt optimization (conditional): boosts retrieval/generation quality for retrieval-like intents.
- 6) Case context enrichment: injects case timeline/risk signals and snapshot continuity.
- 7) High-reasoning selector: optional progressive rollout path for higher-depth reasoning.
- 8) Copilot execution: dispatches to CRUD handler, specialized agent, legal search, or RAG path.
- 9) Memory persistence: stores exchange with metadata for future continuity.
- 10) Structured trace: returns execution metadata so behavior is inspectable, not opaque.

Assistant intent surface (examples)

- Detected intents include: create_case, create_client, create_prompt_library_entry, request_document_upload, request_audio_upload, list_clients, list_cases, delete_case_appointment, update_case_appointment, delete_client, update_client, delete_prompt_library_entry, update_prompt_library_entry, list_prompt_library, delete_case, update_case_status, update_case, create_case_appointment, list_case_appointments, list_case_documents, optimize_prompt, trace_case_evidence, generate_case_memory, monitor_deadlines_case, draft_contract_redline, case_draft_negotiation_strategy.
- These intents never case querying, summaries, risk analysis, evidence tracing, timeline/deadline intelligence, drafting, booking, CRUD actions, and global Q&A.

Why your assistant is already strong

- It is not only a chat bot; it is an intent-driven legal workflow engine with deterministic routing.
- It uses grounded retrieval and can attach source/citation metadata.
- It separates chat mode, legal-search mode, and agent mode, which is good for safety and operator control.
- It is extensible: 21 specialized agent classes already exist to expand legal capabilities quickly.

Backend Deep Dive

Where most project value lives: APIs, services, data, and controls

API and service depth

- Detected API endpoints: 87.
- Top router distribution: rag:17, client_portal:10, intelligence:9, document_router:8, appointments:5, auth:5, cases:5, clients:5, voice:5, prompt_library:4
- Routing domains cover authentication, users/clients/cases, appointments/calls/consultations, document ingestion, evidence review, AI/copilot, search, and voice.
- AI endpoint layer exposes prompt optimization, copilot orchestration, feedback loop, legal search behavior, and workflow execution.

Data and contract architecture

- SQLAlchemy model count: 23 (cases, documents/chunks/entities, voice, consultations, feedback, artifacts, portal auth, jobs, and more).
- Pydantic schema count: 121 (explicit API contracts for request/response typing).
- Case-centric design is a strong fit for legal operations: documents, deadlines, evidence, voice, assistant memory, and generated outputs stay connected to matter context.

Backend quality characteristics

- Multi-tenant boundaries are handled in query scope and auth dependencies.
- Request-ID and process-time middleware improves observability and incident debugging.
- Startup schema patches support smooth evolution without manual emergency migration steps.
- Background jobs keep long-running operations off request latency paths.
- AI service module footprint (40 modules) indicates a mature, decomposed backend rather than a monolith.

Frontend Summary (short)

Your UI layer is intentionally lighter than backend complexity

Internal frontend (frontend/src)

- React + TypeScript workspace provides case-centric command surfaces for assistant interaction and legal operations.
- Feedback capture, mode switching, and typed API clients are integrated.
- Frontend API usage count indicates broad backend feature coverage: 27 mapped client calls.

Client portal (client-portal/src)

- Separate portal keeps public intake/status workflows isolated from internal legal operations.
- Portal API call map size: 10.
- This separation is good for security posture and cleaner product boundaries.

Why this frontend split is good

- You avoid mixing privileged staff workflows with public client interactions.
- You can evolve internal UX fast without breaking client-facing intake behavior.
- Typed API contracts reduce integration drift as backend evolves.

How We Make It Better

Concrete roadmap to raise reliability, legal trust, and maintainability

Assistant roadmap

- Add stronger intent confidence telemetry dashboards (misroute trend by intent/jurisdiction).
- Expand high-reasoning rollout with explicit SLO guardrails before broader percentage increase.
- Increase evidence-grounding checks for every high-impact response path.
- Add richer fallback UX so users immediately know what to correct when confidence is low.

Backend roadmap (priority)

- Raise automated test depth around orchestration transitions and permission boundaries.
- Add domain-level health SLOs (copilot latency, grounding coverage, fallback rate, job queue lag).
- Expand architecture docs alongside code changes so non-technical operators can onboard quickly.
- Continue decomposing heavy services into smaller policy modules where complexity grows.
- Add migration governance checks to keep schema evolution safe under rapid AI-driven development.

Frontend roadmap (secondary)

- Continue polishing explanation UX around assistant confidence, reasons, and citations.
- Improve guided flows for new users to reduce dependency on implicit AI operation knowledge.
- Add compact in-product architecture/help overlays linked to live backend capabilities.

Appendix A: Agent Catalog (1/1)

All detected AI agents and public methods

agent_output_formatter [agent_output_formatter.py]: normalize_text, sanitize_text, extract_json_payload, normalize_string_list, normalize_date_items, build_quality_guidance

BookingAgent [booking_agent.py]: analyze_consultations

CaseMemoryAgent [case_memory_agent.py]: build_case_memory

CaseReasoningAgent [case_reasoning_agent.py]: analyze_case

ContractRedlineAgent [contract_redline_agent.py]: draft_redline

copilot_intent_execution_agent [copilot_intent_execution_agent.py]: execute

DeadlineObligationAgent [deadline_obligation_agent.py]: monitor_deadlines

DocumentComparisonAgent [document_comparison_agent.py]: compare_case_documents

DraftingAgent [drafting_agent.py]: draft_client_update_email

EvidenceStrengthAgent [evidence_strength_agent.py]: evaluate_evidence_strength

EvidenceTraceAgent [evidence_trace_agent.py]: build_claim_trace

InsightAgent [insight_agent.py]: generate_case_insights

IntakeAgent [intake_agent.py]: process_transcript

NegotiationStrategyAgent [negotiation_strategy_agent.py]: draft_strategy

PromptCorrectionAgent [prompt_correction_agent.py]: correct_query

PromptOptimizerAgent [prompt_optimizer_agent.py]: optimize_query

RetrievalAgent [retrieval_agent.py]: retrieve

summarization_agent [summarization_agent.py]: available, summarize_document

TimelineAgent [timeline_agent.py]: build_case_timeline

VerifierAgent [verifier_agent.py]: verify_answer

VisionAnalysisAgent [vision_analysis_agent.py]: analyze

Appendix B: Endpoint Inventory (1/2)

Detected API surface across backend routers

```
[appointments]
  GET    /calendar/case/{case_id}
  POST   /calendar/case/{case_id}
  GET    /calendar/me
  DELETE /calendar/{appointment_id}
  PUT    /calendar/{appointment_id}
[auth]
  POST   /auth/invites
  POST   /auth/login
  GET    /auth/me
  PUT    /auth/me
  POST   /auth/register
[calls]
  GET    /calls/case/{case_id}
  POST   /calls/case/{case_id}
  GET    /calls/{call_session_id}
[cases]
  GET    /cases
  POST   /cases
  DELETE /cases/{case_id}
  GET    /cases/{case_id}
  PUT    /cases/{case_id}
[client_portal]
  POST   /portal/assistant
  POST   /portal/auth/login
  POST   /portal/auth/login/request-code
  POST   /portal/auth/login/verify-code
  GET    /portal/auth/me
  POST   /portal/auth/register
  POST   /portal/cases/{case_id}/uploads
  GET    /portal/dashboard
  POST   /portal/intake/submit
  GET    /portal/jobs/{job_id}
[clients]
  GET    /clients
  POST   /clients
  DELETE /clients/{client_id}
  GET    /clients/{client_id}
  PUT    /clients/{client_id}
[consultations]
  GET    /consultations/case/{case_id}
  POST   /consultations/from-recording/{recording_id}
[document_router]
  GET    /documents/case/{case_id}
  GET    /documents/case/{case_id}/image-batches
  GET    /documents/image-assets/{asset_id}/file
```

Appendix B: Endpoint Inventory (2/2)

Detected API surface across backend routers

```
  PUT    /prompt-library/{entry_id}
[public]
  POST   /public/intake/submit
  GET    /public/intake/{public_reference}
  POST   /public/tenants/{tenant_slug}/intake/submit
[rag]
  POST   /ai/agent-workflow
  GET    /ai/artifacts/versions
  POST   /ai/artifacts/versions/agent-revise
  POST   /ai/artifacts/versions/edit
  POST   /ai/artifacts/versions/{version_id}/select
  POST   /ai/ask
  POST   /ai/cases/{case_id}/snapshot/refresh
  POST   /ai/copilot
  POST   /ai/feedback
  GET    /ai/feedback/weekly-summary
  GET    /ai/jobs/{job_id}
  POST   /ai/optimize-prompt
  POST   /ai/process-document/{document_id}
  GET    /ai/provider-status
  POST   /ai/search
  POST   /ai/test-llm
  POST   /ai/translate
[search]
  GET    /search/documents
[sources]
  GET    /users
[users]
  GET    /users
[voice]
  GET    /voice/case/{case_id}
  POST   /voice/transcribe
  POST   /voice/upload
  GET    /voice/{recording_id}
  GET    /voice/{recording_id}/file
```

Appendix C: Data and Contracts

Models, schemas, intents, and AI modules

SQLAlchemy models

- Appointment, BackgroundJob, CallSession, Case, CaseContextSnapshot, CaseImageAsset, CaseMemoryEntry, Client, ClientPortalAccount, ClientPortalLoginCode, ConsultationRequest, CopilotFeedback, Document, DocumentChunk, DocumentEntity, EvidenceAnalysisReview, GeneratedArtifactVersion, ImageDocumentBatch, PromptLibraryEntry, StaffInvite, Tenant, User, VoiceRecording

Pydantic schemas

- AppointmentCreate, AppointmentUpdate, AppointmentOut, AppointmentActionResponse, CallSessionCreate, WhatsAppConsentWebhookRequest, CallSessionOut, CallSessionCreateResponse, CaseCreate, CaseUpdate, CaseOut, ClientPortalRegisterRequest, ClientPortalLoginRequest, ClientPortalToken, ClientPortalMessageResponse, ClientPortalLoginCodeRequest, ClientPortalLoginCodeVerifyRequest, ClientPortalAccountOut, ClientPortalConsultationItem, ClientPortalCaseItem, ClientPortalDocumentItem, ClientPortalActivityItem, ClientPortalCalendarItem, ClientPortalDashboardMetrics, ClientPortalDashboardResponse, ClientPortalAssistantRequest, ClientPortalAssistantResponse, ClientCreate, ClientUpdate, ClientOut, ConsultationRequestOut, ConsultationFromTranscriptResponse, DocumentChunkOut, DocumentOut, DocumentListItemOut, DocumentAIProcessingOut, DocumentUploadResponse, CaseImageAssetOut, ImageDocumentBatchOut, EvidenceAnalysisReviewOut, ImageBatchUploadResponse, ImageBatchDetailResponse, EvidenceReviewDecisionRequest, EvidenceReviewListResponse, AskRequest, SearchRequest, CopilotRequest, RetrievedChunk, RetrievalSearchResponse, SourceItem, AskResponse, CopilotResponse, EntityOut, ProcessDocumentResponse, ImportantDateItem, DocumentSummaryOut, StructuredDocumentInsightsOut, DocumentSummarizeResponse, FullDocumentAnalysisOut, SearchResultItem, IntelligenceSearchResponse, ReviewTableRowOut, CaseReviewTableResponse, PromptLibraryCreate, PromptLibraryUpdate, PromptLibraryOut, PublicIntakeSubmitResponse, PublicIntakeStatusResponse, AskRequest, SearchRequest, ConversationTurn, CopilotAttachment, CopilotRequest, AgentWorkflowRequest, LLMTestRequest, SemanticTranslateRequest, PromptOptimizationRequest, PromptOptimizationResponse, RetrievedChunk, SearchResponse, SourceItem, CitationItem, CacheMetadata, BackgroundJobItem, ArtifactVersionOut, ArtifactContext, JurisdictionContext, VisionField, VisionCitationItem, VisionAuthenticityReview, VisionResult, AskResponse, CopilotResponse, ReasoningCandidateScore, ReasoningCandidateResult, HighReasoningResult, CopilotFeedbackCreateRequest, CopilotFeedbackOut, CopilotFeedbackWeeklyItem, CopilotFeedbackWeeklySummaryResponse, WorkflowStage, AgentWorkflowResponse, ProviderStatusResponse, LLMTestResponse, SemanticTranslateResponse, ArtifactVersionListResponse, ArtifactVersionManualEditRequest, ArtifactVersionAgentReviseRequest, ArtifactVersionMutationResponse, TenantCreate, TenantOut, UserRegister,